Node.js Software Protection & Node-Locking Strategy

Technical Implementation Specification & Operational Playbook

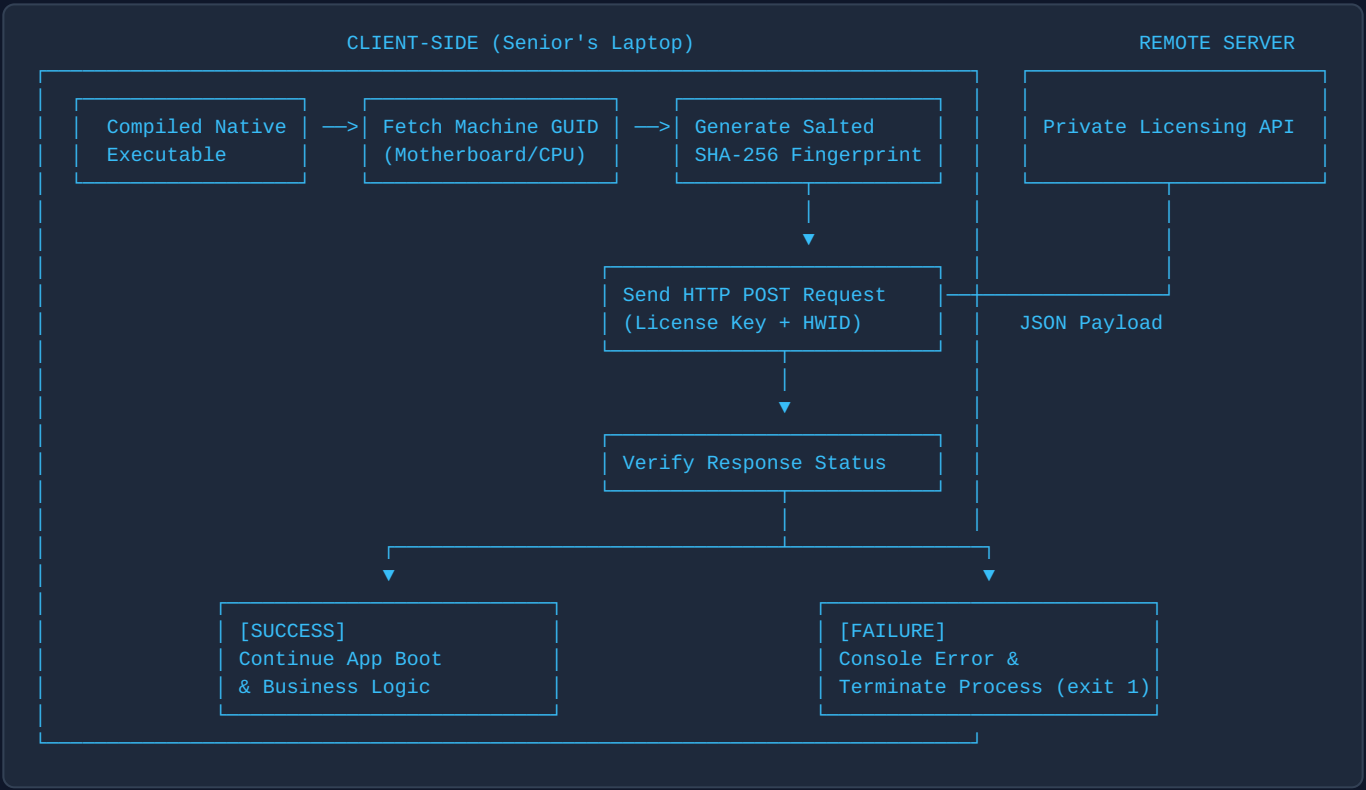
Version: 1.0 **Architecture:** Node.js Backend **Deployment:** On-Premise Binary

Executive Summary

When deploying a proprietary Node.js application to third-party hardware, standard source code protection measures (such as removing `.git` repositories) are insufficient. Source files left on local drives can be easily copied, modified, or re-distributed.

This document details an end-to-end strategy to protect your intellectual property by implementing **Hardware-Bound Licensing (Node-Locking)**, **Code Obfuscation**, and **Native Binary Compilation**.

Architecture Overview



Section 1: Hardware Fingerprinting & License Validation

1. Requirements

Install the necessary hardware extraction and HTTP utility packages in your project root:

```
npm install node-machine-id axios
```

2. Licensing Guard Implementation

Create `licensing.js` in your source directory. This module extracts system GUIDs, hashes them with a private salt, and verifies authorization against a remote server.

```
// licensing.js
const { machineIdSync } = require('node-machine-id');
const crypto = require('crypto');
const axios = require('axios');

const CONFIG = {
  LICENSE_KEY: "KEY-SENIOR-PROD-01",
  LICENSE_SERVER: "https://your-license-server.com/api/v1/verify",
  SECRET_SALT: "YOUR_CUSTOM_PROJECT_SALT_2026", // Unique project identifier
  TIMEOUT_MS: 4000
};

/**
 * Reads hardware UUID and generates a salted HMAC-SHA256 fingerprint.
 */
function getHardwareFingerprint() {
  try {
    const rawMachineId = machineIdSync({ original: true });
    return crypto
      .createHmac('sha256', CONFIG.SECRET_SALT)
      .update(rawMachineId)
      .digest('hex');
  } catch (err) {
    console.error("FATAL: Failed to retrieve system hardware fingerprint.");
    process.exit(1);
  }
}

/**
 * Validates the hardware fingerprint against the central licensing API.
 */
async function verifyLicense() {
  const hardwareId = getHardwareFingerprint();

  try {
    const response = await axios.post(
      CONFIG.LICENSE_SERVER,
      {
        licenseKey: CONFIG.LICENSE_KEY,
        hardwareId: hardwareId
      },
      { timeout: CONFIG.TIMEOUT_MS }
    );

    if (response.data && response.data.valid === true) {
      console.log("License successfully verified.");
      return true;
    } else {
      console.error(" [31m%s [0m", `License Error: ${response.data.message || "Unauthorized machine."}
`);
      process.exit(1);
    }
  } catch (error) {
    console.error(" [31m%s [0m", "License Check Failed: Validation server unreachable.");
    process.exit(1);
  }
}

module.exports = { verifyLicense };
```

3. Application Startup Integration

Invoke the licensing guard at the root execution entry point (`index.js`). No application server or business logic must execute prior to this check.

```
// index.js
const { verifyLicense } = require('./licensing');

async function bootstrap() {
  // Enforce hardware lock prior to initialization
  await verifyLicense();

  // --- BEGIN PROTECTED APPLICATION LOGIC ---
  const express = require('express');
  const app = express();

  app.get('/', (req, res) => res.json({ status: "Active", authorization: "Granted" }));

  app.listen(3000, () => {
    console.log("Application running securely on port 3000.");
  });
}

bootstrap();
```

Section 2: Code Obfuscation & Binary Compilation

To prevent unauthorized modification or removal of `licensing.js`, plain JavaScript source code must not be transferred to the target machine.

1. Build Tools Installation

```
npm install -D javascript-obfuscator @yao-pkg/pkg
```

2. Code Obfuscation Stage

Run `javascript-obfuscator` across the source codebase to mangle variable names, inject self-defending mechanisms, and encrypt string literals.

```
npx javascript-obfuscator ./src   --output ./dist   --compact true   --self-defending true   --string-array true   --string-array-encoding 'rc4'   --control-flow-flattening true
```

> **Security Flags Explained:**
>
> - `--self-defending true`: Injects anti-tampering logic that crashes the app if formatted or executed via a debugger.
> - `--string-array-encoding 'rc4'`: Encrypts internal API endpoints, salt values, and string variables.
> - `--control-flow-flattening true`: Scrambles code control paths to eliminate readable execution structures.

3. Native Executable Packaging Stage

Pack the obfuscated JavaScript application, dependencies, and the Node.js runtime environment into a single compiled binary file using `pkg`.

```
npx pkg ./dist/index.js --targets node18-win-x64 --output ./release/app-windows.exe
```

Section 3: Remote Licensing Backend Requirements

The remote licensing server manages device registration and access control.

Endpoint Specification

- **Protocol:** HTTPS POST
- **Route:** `/api/v1/verify`
- **Request Body:**

```
{
  "licenseKey": "KEY-SENIOR-PROD-01",
  "hardwareId": "a8f9b2c3d4e5f67890123456789abcdef"
}
```

Verification Flow (Server-Side Logic)

1. **Lookup:** Query database for `licenseKey`.
2. **Status Check:** If `is_active` is `false`, return HTTP 403 (`"License revoked"`).
3. **First-Time Registration:** If stored `hardware_id` is `null`, record the incoming `hardwareId` as the bound machine. Return HTTP 200 (`valid: true`).
4. **Validation:** If stored `hardware_id` matches incoming `hardwareId`, return HTTP 200 (`valid: true`).
5. **Mismatch:** If stored `hardware_id` does not match, return HTTP 403 (`"Unauthorized hardware"`).

Section 4: Deployment & Operational Runbook

Target System Setup Checklist

When setting up the target machine (e.g., Senior's Laptop):

- **Do NOT** clone source repositories or leave raw code on the disk.
- **Do NOT** install Node.js source files or project dependencies directly on the drive.
- **Do NOT** log into GitHub or developer credential accounts.
- **DO** copy only the generated binary (`app-windows.exe`) and necessary assets (e.g., static assets, `.env.production`).
- **DO** execute the binary file to trigger the first-time hardware registration sequence.

Revocation Protocol (Kill Switch Execution)

If unauthorized distribution or usage is identified:

1. Access your remote licensing server database.
2. Set `is_active = false` (or clear the authorized `hardware_id`) for the assigned license key.
3. The next time the client binary is started (or re-validated), authorization will be rejected, and the executable will terminate immediately.

Security Guarantees Matrix

Attack Vector	Defense Mechanism	Risk Level
Copying app to another laptop	Hardware ID mismatch triggers auto-shutdown on startup	Mitigated
Removing .git to clear history	Source code is missing; application runs via compiled C++ wrapper	Mitigated
Editing code to bypass guard	Source files do not exist on disk; binary editing breaks hash integrity	Mitigated
Decompiling or Beautifying JS	Self-defending obfuscation triggers process failure	Mitigated
Unauthorized Long-Term Usage	Central API revocation revokes license key access globally	Mitigated